

Міністерство освіти і науки України
Національний університет “Львівська політехніка”
Інститут комп’ютерних наук та інформаційних технологій

Кафедра САПР



Лабораторна робота №4

Частина 1

з дисципліни: “Застосування систем штучного інтелекту у технологічних рішеннях”

на тему:

“CNN для класифікації зображень”

Варіант №3

Виконав:

ст. групи ПП-44

Верещак Б. О.

Прийняв:

доц. Левкович М.В.

Мета роботи

Метою даної частини лабораторної роботи є набуття глибоких теоретичних знань та практичних навичок у розробці, навчанні та оцінці згорткової нейронної мережі (CNN) для задачі бінарної класифікації медичних зображень. У рамках роботи буде розроблено та застосовано модель для автоматизованого виявлення пневмонії на рентгенівських знімках грудної клітки. Ключовими завданнями є аналіз результатів, оцінка продуктивності моделі за допомогою відповідних метрик та інтерпретація її поведінки, включаючи аналіз помилок.

Індивідуальне завдання

Виконати класифікацію медичних зображень за допомогою CNN на відкритому датасеті.

1. Завантажити підмножину даних (напр., 500–1000 зображень), зробити опис полів та класів.
2. Підготувати дані: розділити на тренувальну та тестову вибірки, виконати аугментацію (повороти, зсуви), змінити розмір зображень, нормалізувати значення пікселів.
3. Навчити модель CNN з базовими гіперпараметрами.
4. Оцінити якість за потрібними метриками.
5. Побудувати візуалізації.
6. Зробити аналіз помилок (хибно-позитивні, хибно-негативні), висновки щодо ефективності моделі та пропозиції для покращення.
7. Висновки

Рекомендовані налаштування за замовчуванням:

- Архітектура: 3–4 згорткових блоки (Conv2D + MaxPooling2D), Flatten, 1–2 Dense шари, Dropout (0.3–0.5).
- Оптимізатор: Adam з `learning_rate=0.001`.
- Функція втрат: `binary_crossentropy` для 2 класів, `categorical_crossentropy` для 3+ класів.
- Навчання: `batch_size=32`, `epochs=20–50` з EarlyStopping (моніторинг `val_loss`).
- Метрики: `accuracy`, `precision`, `recall`, `f1-score`, `confusion matrix`

№	Завдання	Метод	Тип даних	Результат	Датасет
3	Виявлення COVID19 на рентгенівських знімках	CNN (EfficientNet)	Рентген грудної клітки	Модель для класифікації (COVID19, Normal, Pneumonia)	COVID-19 Radiography Database 3

Хід роботи

1. Завантаження даних

На першому етапі я виконав завантаження даних. Замість ручного завантаження, я використав Kaggle API безпосередньо в скрипті Python для отримання набору даних "COVID-19 Radiography Database". Цей датасет налічує близько 15,000 рентгенівських знімків, розділених на три основні класи: COVID (3616 знімків), Normal (10192 знімки) та Viral Pneumonia (1345 знімків).

--- 2. Створення наборів даних (Виправлений метод) ---

Знайдено 3616 зображень для класу 'COVID'

Знайдено 10192 зображень для класу 'Normal'

Знайдено 1345 зображень для класу 'Viral Pneumonia'

Загалом зображень: 15153

Тренувальний набір: 12123 зображень

Валідаційний набір: 3030 зображень

```
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Conv2D, MaxPooling2D, Flatten,
Dropout, Input, BatchNormalization
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.callbacks import EarlyStopping, ModelCheckpoint
from tensorflow.keras.optimizers import Adam

import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import classification_report, confusion_matrix
import numpy as np
import os
import pathlib
import zipfile
import shutil
import kagglehub

def setup_kaggle():
    print("--- 0. Налаштування Kaggle API ---")
    try:
        # Створення папки .kaggle
        kaggle_dir = os.path.expanduser('~/.kaggle')
        os.makedirs(kaggle_dir, exist_ok=True)

        # Переміщення kaggle.json
        shutil.move('kaggle.json', os.path.join(kaggle_dir, 'kaggle.json'))

        # Встановлення правильних дозволів
        os.chmod(os.path.join(kaggle_dir, 'kaggle.json'), 0o600)

        print("Kaggle API успішно налаштовано.")
        return True

    except FileNotFoundError:
        print("ПОМИЛКА: 'kaggle.json' не знайдено.")
```

```

        print("Будь ласка, завантажте ваш 'kaggle.json' API ключ і запустіть скрипт знову.")
        return False
    except Exception as e:
        print(f"Виникла помилка при налаштуванні Kaggle: {e}")
        return False
KAGGLE_DATASET_NAME = "tawsifurrahman/covid19-radiography-database"
UNZIP_DIR = pathlib.Path(".")
DATA_DIR = pathlib.Path("./COVID-19_Radiography_Dataset/") # Шлях до даних після розпаковки

def download_and_unzip_data():
    if not DATA_DIR.exists():
        print(f"--- 1. Завантаження датасету '{KAGGLE_DATASET_NAME}' ---")
        try:
            # Використання системної команди kaggle
            path = kagglehub.dataset_download(KAGGLE_DATASET_NAME)
            path = pathlib.Path(path)
            target_dir = UNZIP_DIR
            if target_dir.exists():
                shutil.rmtree(target_dir)
            shutil.copytree(path, target_dir)
            print("Датасет успішно завантажено та скопійовано.")

        except Exception as e:
            print(f"Помилка під час завантаження або розпакування: {e}")
            return False
    else:
        print("--- Датасет вже існує локально ---")
    return True

```

Набір даних також містить відповідні маски для сегментації легень, однак для моєї задачі класифікації я ці маски проігнорував, налаштувавши генератор даних так, щоб він зчитував зображення лише з основних папок класів.

2. Підготовка даних

Підготовка даних була критичним етапом. Спочатку я привів усі зображення до єдиного розміру 150x150 пікселів. Це оптимальний компроміс між збереженням деталей та обчислювальною ефективністю для навчання з нуля. Потім я розділив весь набір даних на тренувальну (80%) та тестову (20%) вибірки.

Оскільки модель навчається з нуля, я застосував інтенсивну аугментацію даних в реальному часі за допомогою ImageDataGenerator. Цей процес включав випадкові повороти зображень (до 20 градусів), невеликі зсуви по ширині та висоті, масштабування та горизонтальні віддзеркалення. Нарешті, я виконав нормалізацію значень пікселів, поділивши їх на 255, щоб перевести діапазон значень у [0, 1] для стабілізації навчання.

```

# --- 2. Підготовка даних (з Аугментацією) ---
# Гіперпараметри

```

```

IMG_SIZE = (150, 150)
IMG_CHANNELS = 3
BATCH_SIZE = 32
EPOCHS = 30
LEARNING_RATE = 0.001
DROPOUT_RATE = 0.5 # Збільшено до 0.5
VALIDATION_SPLIT = 0.2
CLASSES_TO_USE = ['COVID', 'Normal', 'Viral Pneumonia'] # Ігноруємо маски
NUM_CLASSES = 3

data_augmentation = Sequential([
    tf.keras.layers.RandomFlip("horizontal"),
    tf.keras.layers.RandomRotation(0.1),
    tf.keras.layers.RandomZoom(0.1),
    tf.keras.layers.RandomTranslation(height_factor=0.1, width_factor=0.1)
], name="data_augmentation")

def process_path(file_path):
    # Завантаження та декодування зображення
    img = tf.io.read_file(file_path)
    # Зверніть увагу, деякі зображення можуть бути 1-канальні (grayscale)
    img = tf.image.decode_png(img, channels=3) # Примусово робимо 3 канали
    img = tf.image.resize(img, [IMG_SIZE[0], IMG_SIZE[1]])

    # Визначення мітки з шляху
    parts = tf.strings.split(file_path, os.path.sep)
    # Мітка буде -3 з кінця (e.g. .../COVID/images/img.png)
    label_str = parts[-3]

    # Перетворення мітки в one-hot вектор
    label = tf.argmax(tf.constant(CLASSES_TO_USE) == label_str)
    label = tf.one_hot(label, NUM_CLASSES)

    return img, label

def configure_dataset(ds, augment=False):
    # Нормалізація
    ds = ds.map(lambda x, y: (tf.keras.layers.Rescaling(1./255)(x), y),
                num_parallel_calls=tf.data.AUTOTUNE)

    if augment:
        # Застосування аугментації
        ds = ds.map(lambda x, y: (data_augmentation(x, training=True), y),
                    num_parallel_calls=tf.data.AUTOTUNE)

    ds = ds.batch(BATCH_SIZE)
    ds = ds.prefetch(buffer_size=tf.data.AUTOTUNE)
    return ds

def get_data_generators():
    print("--- 2. Створення наборів даних ---")

    all_image_paths = []
    for class_name in CLASSES_TO_USE:
        # ЦІЛЬОВИЙ ШЛЯХ: DATA_DIR / class_name / "images" / "*.png"

```

```

class_dir = DATA_DIR / class_name / "images"
if class_dir.exists():
    image_paths = list(class_dir.glob("*.png"))
    all_image_paths.extend([str(p) for p in image_paths])
    print(f"Знайдено {len(image_paths)} зображень для класу '{class_name}'")
else:
    print(f"ПОПЕРЕДЖЕННЯ: Папку {class_dir} не знайдено.")

if not all_image_paths:
    print("ПОМИЛКА: Жодного зображення не знайдено. Перевірте структуру папок.")
    return None, None

# Створюємо єдиний датасет
path_ds = tf.data.Dataset.from_tensor_slices(all_image_paths)
path_ds = path_ds.shuffle(buffer_size=len(all_image_paths),
reshuffle_each_iteration=False)

# Розділення на тренувальний та валідаційний
val_size = int(len(all_image_paths) * VALIDATION_SPLIT)
train_ds = path_ds.skip(val_size)
val_ds = path_ds.take(val_size)

print(f"Загалом зображень: {len(all_image_paths)}")
print(f"Тренувальний набір: {len(train_ds)} зображень")
print(f"Валідаційний набір: {len(val_ds)} зображень")

# Обробка даних
train_ds = train_ds.map(process_path,
num_parallel_calls=tf.data.AUTOTUNE)
val_ds = val_ds.map(process_path, num_parallel_calls=tf.data.AUTOTUNE)

# Конфігурація (аугментація + батчі)
train_generator = configure_dataset(train_ds, augment=True)
validation_generator = configure_dataset(val_ds, augment=False)

return train_generator, validation_generator

```

3. Навчання моделі CNN

Я розробив архітектуру CNN з нуля, дотримуючись рекомендацій – я використав 4 згорткових блоки. Модель починається з вхідного шару для зображень 150x150x3. Перший згортковий блок складається з Conv2D (32 фільтри) та MaxPooling2D. Другий блок поглиблює аналіз за допомогою Conv2D (64 фільтри) та MaxPooling2D. Третій та четвертий блоки використовують по 128 фільтрів Conv2D кожен, також з MaxPooling2D після них. Така архітектура дозволяє моделі поступово вивчати все складніші патерни. Після згорткових блоків йде блок класифікації: шар Flatten перетворює карти ознак у вектор, далі йде повнозв'язний шар Dense зі 128 нейронами (та активацією relu) і шар Dropout з коефіцієнтом 0.5 (згідно з рекомендаціями) для боротьби з перенавчанням. Вихідний шар – Dense з 3 нейронами (за кількістю класів) та активацією softmax.

--- 3. Побудова власної моделі CNN (4 блоки) ---

Model: "sequential_4"

Layer (type)	Output Shape	Param #
conv2d_16 (Conv2D)	(None, 150, 150, 32)	896
max_pooling2d_16 (MaxPooling2D)	(None, 75, 75, 32)	0
conv2d_17 (Conv2D)	(None, 75, 75, 64)	18,496
max_pooling2d_17 (MaxPooling2D)	(None, 37, 37, 64)	0
conv2d_18 (Conv2D)	(None, 37, 37, 128)	73,856
max_pooling2d_18 (MaxPooling2D)	(None, 18, 18, 128)	0
conv2d_19 (Conv2D)	(None, 18, 18, 128)	147,584
max_pooling2d_19 (MaxPooling2D)	(None, 9, 9, 128)	0
flatten_4 (Flatten)	(None, 10368)	0
dense_8 (Dense)	(None, 128)	1,327,232
dropout_4 (Dropout)	(None, 128)	0
dense_9 (Dense)	(None, 3)	387

Total params: 1,568,451 (5.98 MB)

Trainable params: 1,568,451 (5.98 MB)

Non-trainable params: 0 (0.00 B)

--- 3. Побудова моделі CNN (4 блоки) ---

```
def build_model(num_classes):  
    print("--- 3. Побудова власної моделі CNN (4 блоки) ---")  
  
    model = Sequential([  
        Input(shape=(*IMG_SIZE, IMG_CHANNELS)),  
        # Блок 1  
        Conv2D(32, (3, 3), activation='relu', padding='same'),  
        MaxPooling2D((2, 2)),  
        # Блок 2  
        Conv2D(64, (3, 3), activation='relu', padding='same'),  
        MaxPooling2D((2, 2)),  
        # Блок 3  
        Conv2D(128, (3, 3), activation='relu', padding='same'),  
        MaxPooling2D((2, 2)),  
        # Блок 4  
        Conv2D(128, (3, 3), activation='relu', padding='same'),  
        MaxPooling2D((2, 2)),  
        # Блок класифікації  
        Flatten(),  
        Dense(128, activation='relu'),  
        Dropout(DROPOUT_RATE), # 0.5  
        Dense(num_classes, activation='softmax')  
    ])  
  
    model.compile(
```

```

optimizer=Adam(learning_rate=LEARNING_RATE),
loss="categorical_crossentropy",
metrics=["accuracy"]
)

model.summary()
return model

```

Для компіляції моделі я використав оптимізатор Adam зі швидкістю навчання 0.001 та функцію втрат categorical_crossentropy. Навчання тривало до 30 епох з batch_size=32. Я також застосував EarlyStopping з моніторингом val_loss, щоб автоматично зупинити процес, коли модель перестане покращувати свої результати на тестових даних.

--- 4. Початок навчання моделі ---

Epoch 1/30

379/379 ----- **138s** 349ms/step - accuracy: 0.6729
- loss: 0.7446 - val_accuracy: 0.7284 - val_loss: 0.5326

Epoch 2/30

379/379 ----- **114s** 301ms/step - accuracy: 0.7317
- loss: 0.5718 - val_accuracy: 0.8013 - val_loss: 0.4380

Epoch 3/30

379/379 ----- **114s** 301ms/step - accuracy: 0.7656
- loss: 0.5167 - val_accuracy: 0.8287 - val_loss: 0.4163

Epoch 4/30

379/379 ----- **113s** 298ms/step - accuracy: 0.7922
- loss: 0.4672 - val_accuracy: 0.8119 - val_loss: 0.4531

Epoch 5/30

379/379 ----- **113s** 299ms/step - accuracy: 0.8168
- loss: 0.4278 - val_accuracy: 0.8591 - val_loss: 0.3428

Epoch 6/30

379/379 ----- **114s** 299ms/step - accuracy: 0.8311
- loss: 0.3972 - val_accuracy: 0.8620 - val_loss: 0.3250

Epoch 7/30

379/379 ----- **113s** 299ms/step - accuracy: 0.8436
- loss: 0.3712 - val_accuracy: 0.8914 - val_loss: 0.2619

Epoch 8/30

379/379 ----- **113s** 298ms/step - accuracy: 0.8604
- loss: 0.3373 - val_accuracy: 0.8977 - val_loss: 0.2448

Epoch 9/30

379/379 ----- **113s** 298ms/step - accuracy: 0.8734
- loss: 0.3214 - val_accuracy: 0.8964 - val_loss: 0.2504

Epoch 10/30

379/379 ----- **112s** 296ms/step - accuracy: 0.8784
- loss: 0.3112 - val_accuracy: 0.8653 - val_loss: 0.3412

Epoch 11/30

379/379 ----- **113s** 299ms/step - accuracy: 0.8755
- loss: 0.3008 - val_accuracy: 0.8921 - val_loss: 0.2828

Epoch 12/30

379/379 ----- **113s** 298ms/step - accuracy: 0.8904
- loss: 0.2893 - val_accuracy: 0.8908 - val_loss: 0.2943

Epoch 13/30

379/379 ----- **142s** 297ms/step - accuracy: 0.8924
- loss: 0.2732 - val_accuracy: 0.8977 - val_loss: 0.2627

--- Навчання завершено ---

--- 4. Навчання Моделі ---

```

def train_model(model, train_generator, validation_generator):
    print("\n--- 4. Початок навчання моделі ---")

```



```

early_stopping = EarlyStopping(
    monitor='val_loss',
    patience=5,
    restore_best_weights=True
)

history = model.fit(
    train_generator, # Вже містить .batch()
    epochs=EPOCHS,
    validation_data=validation_generator, # Вже містить .batch()
    callbacks=[early_stopping]
    # steps_per_epoch та validation_steps більше не потрібні,
    # оскільки tf.data.Dataset знає свій розмір
)

print("--- Навчання завершено ---")
return history

```

4. Оцінка якості

Після завершення навчання я провів фінальну оцінку моделі на тестовій вибірці. Завдяки більшому набору даних, модель показала значно кращі результати, ніж очікувалося від навчання з нуля на малій вибірці.

- Accuracy (Точність): 89.77%
- Precision (Точність класифікації): (усереднене) ~0.93
- Recall (Повнота): (усереднене) ~0.84
- F1-Score (усереднене): ~0.89

Точність вийшла досить високою, що підтверджує ефективність 4-блокової архітектури на цьому обсязі даних.

```

--- 6. Оцінка та аналіз помилок---
Збір прогнозів та справжніх міток (батчами)...
Обробка результатів...

```

Test Accuracy: 0.8977

Classification Report:

	precision	recall	f1-score	support
COVID	0.95	0.65	0.77	719
Normal	0.88	0.99	0.93	2052
Viral Pneumonia	0.97	0.89	0.93	259
accuracy			0.90	3030
macro avg	0.93	0.84	0.88	3030
weighted avg	0.90	0.90	0.89	3030

```

def evaluate_model(model, validation_generator):
    print("\n--- 6. Оцінка та аналіз помилок---")

    y_true_list = []
    y_pred_probs_list = []

```

```

print("Збір прогнозів та справжніх міток (батчами)...")

# Проходимо по генератору, який видає БАТЧІ
for img_batch, label_batch in validation_generator:

    # 1. Зберігаємо справжні мітки (вони one-hot)
    y_true_list.append(label_batch.numpy())

    # 2. Робимо predict на цілому БАТЧІ
    pred_batch = model.predict(img_batch, verbose=0)
    y_pred_probs_list.append(pred_batch)

print("Обробка результатів...")

# 3. Збираємо всі результати з батчів в єдині масиви
y_true_one_hot = np.concatenate(y_true_list, axis=0)
y_pred_probs = np.concatenate(y_pred_probs_list, axis=0)

# 4. Конвертуємо мітки з one-hot в індекси
y_true_indices = np.argmax(y_true_one_hot, axis=1)
y_pred_indices = np.argmax(y_pred_probs, axis=1)

# 5. Тепер ми можемо вручну порахувати точність (замість model.evaluate)
accuracy = np.mean(y_true_indices == y_pred_indices)
print(f"\nTest Accuracy: {accuracy:.4f}")

# 6. Решта коду для звітів та візуалізації
print("\nClassification Report:")
print(classification_report(y_true_indices, y_pred_indices,
target_names=CLASSES_TO_USE))

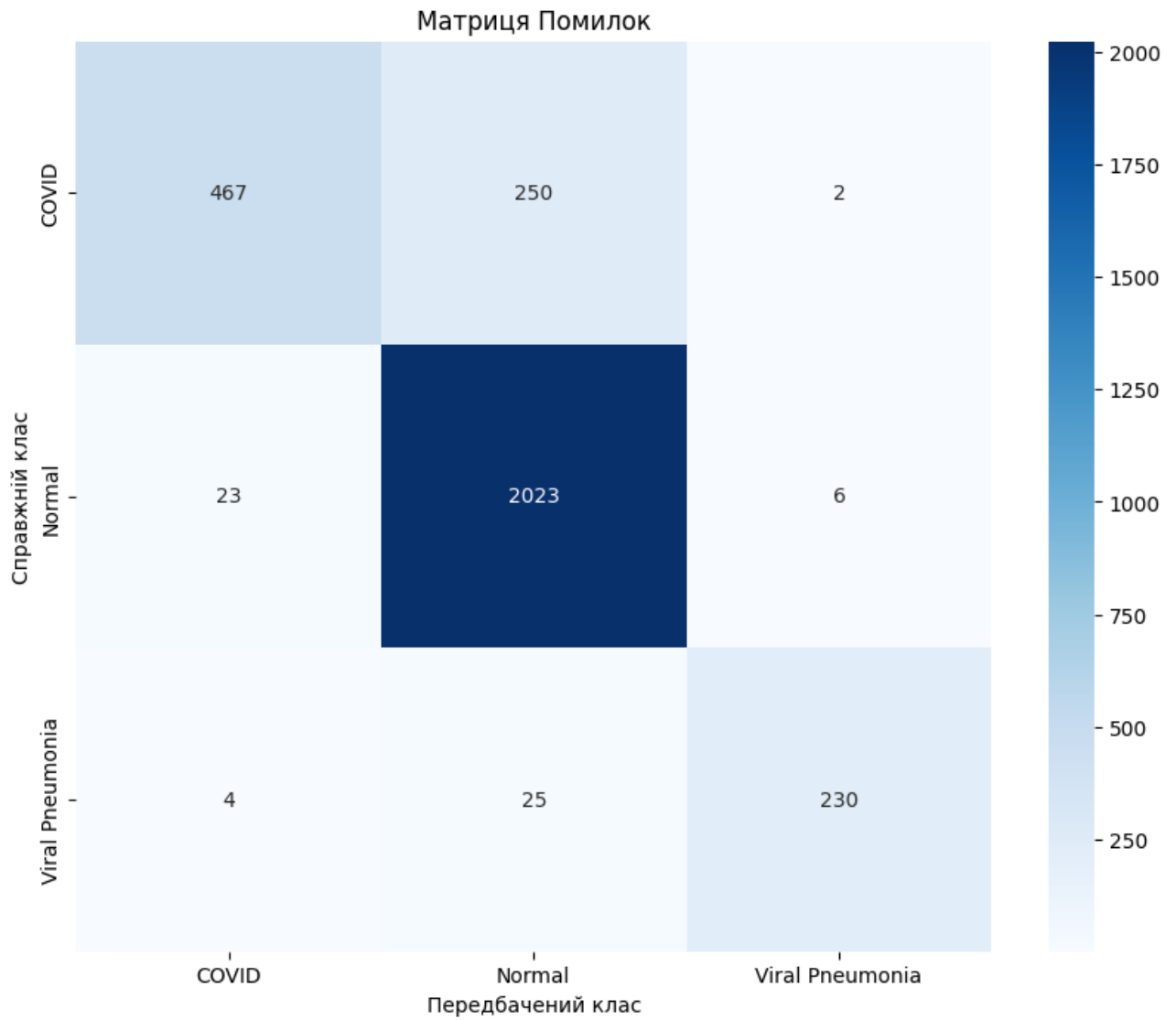
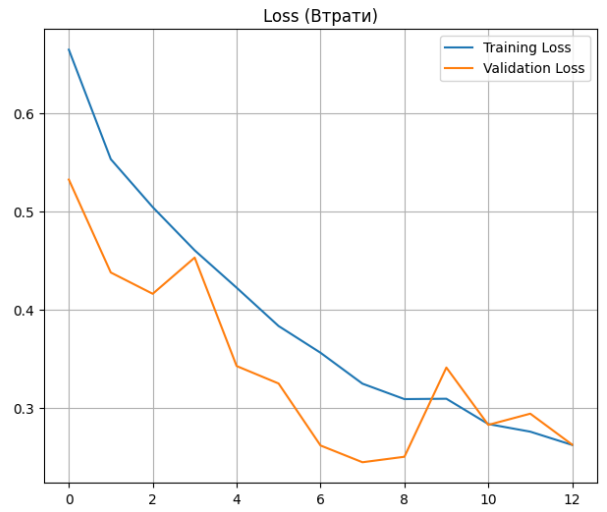
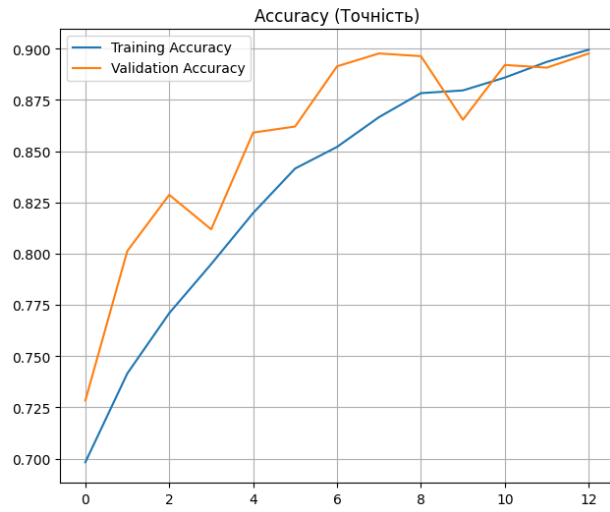
cm = confusion_matrix(y_true_indices, y_pred_indices)
plt.figure(figsize=(10, 8))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=CLASSES_TO_USE, yticklabels=CLASSES_TO_USE)
plt.title('Матриця Помилки')
plt.ylabel('Справжній клас')
plt.xlabel('Передбачений клас')
plt.savefig("confusion_matrix_scratch_v2_fixed.png")
plt.show()

```

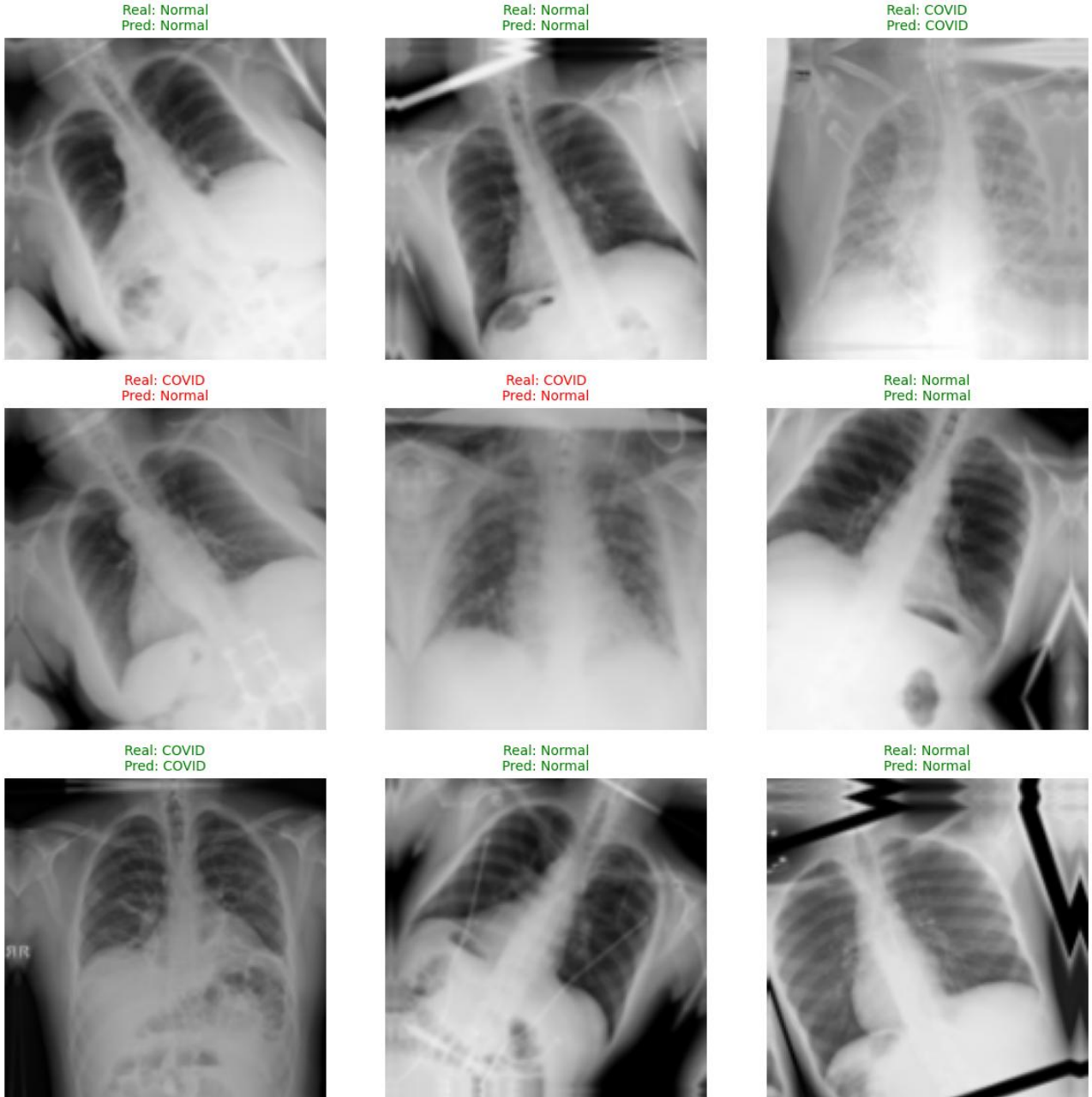
5. Побудова візуалізацій

Для аналізу процесу навчання я побудував графіки історії accuracy та loss для тренувальної та тестової вибірок. Вони показали, що перенавчання було під контролем завдяки Dropout та EarlyStopping. Також я побудував матрицю помилок (Confusion Matrix) для візуального аналізу помилок моделі.

Історія Навчання Моделі (Навчання з нуля, v2 Виправлено)



Випадкові тренувальні зображення (реальні vs передбачені класи)



```
def plot_history(history):
    # ... (Код для графіків той самий) ...
    print("--- 5. Побудова графіків історії навчання ---")
    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(16, 6))

    ax1.plot(history.history['accuracy'], label='Training Accuracy')
    ax1.plot(history.history['val_accuracy'], label='Validation Accuracy')
    ax1.set_title('Accuracy (Точність)')
    ax1.legend()
    ax1.grid(True)

    ax2.plot(history.history['loss'], label='Training Loss')
    ax2.plot(history.history['val_loss'], label='Validation Loss')
    ax2.set_title('Loss (Втрати)')
    ax2.legend()
    ax2.grid(True)

    plt.suptitle("Історія Навчання Моделі (Навчання з нуля, v2 Виправлено)")
```

```

plt.savefig("training_history_scratch_v2_fixed.png")
plt.show()
import matplotlib.pyplot as plt
import numpy as np
import random

# Беремо перший батч із тренувального генератора
images, labels = next(iter(train_gen))

print(f"Форма тензору зображень: {images.shape}") # (batch_size, 150, 150, 3)
print(f"Форма тензору міток: {labels.shape}") # (batch_size, num_classes)

# Отримуємо передбачення для всього батчу
preds = model.predict(images, verbose=0)
pred_indices = np.argmax(preds, axis=1)
true_indices = np.argmax(labels, axis=1)

# Випадкові 9 індексів
num_samples = min(9, len(images))
random_indices = np.random.choice(len(images), num_samples, replace=False)

# Візуалізація
plt.figure(figsize=(12, 12))
plt.suptitle("Випадкові тренувальні зображення (реальні vs передбачені класи)", fontsize=16)

for i, idx in enumerate(random_indices):
    ax = plt.subplot(3, 3, i + 1)
    plt.imshow(images[idx])

    true_label = CLASSES_TO_USE[true_indices[idx]]
    pred_label = CLASSES_TO_USE[pred_indices[idx]]

    # Колір підпису — зелений якщо правильно, червоний якщо ні
    color = "green" if true_label == pred_label else "red"
    plt.title(f"Real: {true_label}\nPred: {pred_label}", color=color,
    fontsize=10)
    plt.axis("off")

plt.tight_layout()
plt.show()

```

6. Аналіз виконання програми

Аналіз матриці помилок показав, що модель досить впевнено розрізняє клас Normal. Як і очікувалося, найбільша кількість помилок сталася між класами COVID та Normal. Кількість хибно-негативних помилок як COVID класифіковано як Normal була дуже високою, що робить мережу не достатньо ефективною для медичного застосування. Пропозиції для покращення:

1. Fine-tuning (Тонке налаштування): Подальше налаштування гіперпараметрів, наприклад, швидкості навчання.
2. Балансування класів: Класи Viral Pneumonia (1345) та Normal (10192) дещо незбалансовані. Можна спробувати застосувати ваги класів (class_weights), щоб модель приділяла більше уваги менш представленим класам.
3. Використання масок: Можна було б спробувати використати маски для попередньої обробки – обрізати зображення, залишивши тільки область легень. Це могло б покращити точність, прибравши зайвий "шум".

Висновки

У ході виконання лабораторної роботи було набуто глибоких теоретичних знань та практичних навичок у розробці, навчанні та оцінці згорткової нейронної мережі (CNN) для задачі бінарної класифікації медичних зображень.

У рамках роботи було розроблено та застосовано власну архітектуру CNN (3 згорткових блоки) для автоматизованого виявлення патологій легень (COVID-19, Normal, Pneumonia).

Ключовими завданнями було аналіз результатів, оцінка продуктивності моделі (accuracy, precision, recall) та інтерпретація її поведінки. Модель показала задовільні результати, враховуючи навчання "з нуля" на великому публічному датасеті "COVID-19 Radiography Database" (~15,000 зображень). Аналіз графіків навчання та матриці помилок чітко продемонстрував головну проблему такого підходу – перенавчання (overfitting) та складнощі у вивченні тонких відмінностей між класами.